Linux Privilege Escalation Automation Toolkit

Detection-only Linux enumeration and security auditing

Red-team enumeration · Blue-team auditing

Author: [Your Name]

Company: [Company]

Supervisor: [Supervisor]

Date: [Date]

Internship project report

Contents

Contents 2
Abstract 4
Introduction 5
Objectives and scope 6
Objectives 6
Scope 6
Tools and technologies 7
Architecture 8
Workflow 9
Implementation 11
Rule catalog 12
Safety model 13
Allowlisted commands 13
Timeout and argument safety 13
No mutation 13
No payloads 13
Runtime proof 13
Lab design 14
Testing and verification 15
Results 16
Mitigation guidance 17
Limitations and future work 18
Current limitations 18
Future work 18
Ethics and legal note 19
Conclusion 20
References 21
Appendix A: Command-line reference 22
Appendix B: JSON structure 23

Abstract

Introduction

Objectives and scope

Tools and technologies

Architecture and workflow

Implementation

Rule catalog

Safety model

Lab design

Testing and verification

Results

Mitigation guidance

Limitations and future work

Ethics and legal note

Conclusion

References

Appendices

Abstract

This project implements a standard-library Python scanner for authorized Linux privilege-boundary audits. It collects identity, SUID/SGID, filesystem permissions, sudo policy, cron jobs, systemd units, file capabilities, and kernel advisory evidence. Explainable rules map collected facts to findings with severity, confidence, mitigation, and references. The scanner is detection-only: it does not execute discovered binaries, modify system files, or change permissions. The report includes JSON, Markdown, HTML, command records, coverage status, and comparison output. In the current build, real Linux scan evidence is SKIPPED; no Linux result totals are reported.

The host-side fixture suite tests portable parsers, rules, report generation, comparison, CLI behavior, and command-runner safety. Linux-only steps are kept separate and marked skipped when the VM cannot provide an authenticated guest session. This distinction keeps test evidence traceable and prevents demo values from being presented as real scan results.

Build status	Observed result
Host platform	Windows; scanner correctly reports Linux-only for scan action
Fixture suite	28 passed, 0 failed, 1 skipped in run 1
Kali VM access	Guest Additions control session unavailable before guest login
Linux lab and real screenshots	SKIPPED until an authenticated Linux session is available

Introduction

Privilege escalation occurs when a process or account gains access beyond its intended authority. Linux permission bits, privileged programs, service definitions, scheduled jobs, sudo policy, and file capabilities all influence that boundary. A weak configuration can let a local account affect a task that later runs with elevated privileges.

Auditing needs to distinguish evidence collection from exploitation. This toolkit records file metadata and configuration text, then applies transparent rules. It describes the possible impact and a defensive correction without testing the weakness by executing it. Administrators should validate every result against the operating system vendor and the system's intended configuration.

The project is designed for an internship demonstration. Its fixtures make parser and rule behavior reviewable on Windows, while real scans require a Linux filesystem and Linux command behavior. The included disposable lab is isolated from the scanner and tracks every item it creates.

Objectives and scope

Objectives

- Collect high-value Linux privilege-boundary configuration with read-only operations.
- Explain each finding in plain language and attach a specific mitigation and reference.
- Record which checks ran and which checks were skipped with a reason.
- Compare two reports by stable rule and evidence fingerprints.
- Prove the safe command boundary with unit tests and before/after file hashes.

Scope

The implementation covers local enumeration only. It does not exploit a weakness, provide proof-of-concept payloads, change system settings, install software, or contact remote targets. Network services, application-layer flaws, cloud permissions, and distribution-specific kernel patch databases are outside this version's scope.

Full filesystem discovery uses GNU find permission predicates and prunes pseudo filesystems. A fast mode limits traversal to common system, application, home, and temporary directories. WSL Windows-drive mounts are excluded. The report identifies limited-account visibility and version-only kernel uncertainty.

Tools and technologies

Technology	Role	Dependency
Python 3.10+ standard library	CLI, collectors, parsing, rules, serialization	Runtime
Bash	Linux wrapper and disposable lab lifecycle	Linux host
pytest	Fixture, safety, CLI, and Linux integration tests	Development only
python-docx	Editable internship report	Documentation only
python-pptx	Editable 15-slide presentation and speaker notes	Documentation only
Pillow and matplotlib	Terminal previews and architecture diagrams	Documentation only
GNU find, getcap, sudo, systemctl, crontab	Read-only Linux enumeration sources	Optional host tools; missing tools are skipped

Direct development package versions are pinned in requirements-dev.txt. The scanner itself imports no third-party runtime library. Development dependencies are installed only inside a virtual environment.

Architecture

Collectors gather raw text and metadata. Parsers convert that material to small dictionaries and lists. Analysis rules are pure functions over those records. The report layer serializes one ScanResult into JSON, Markdown, and HTML. The command runner is the only place that invokes an external program.

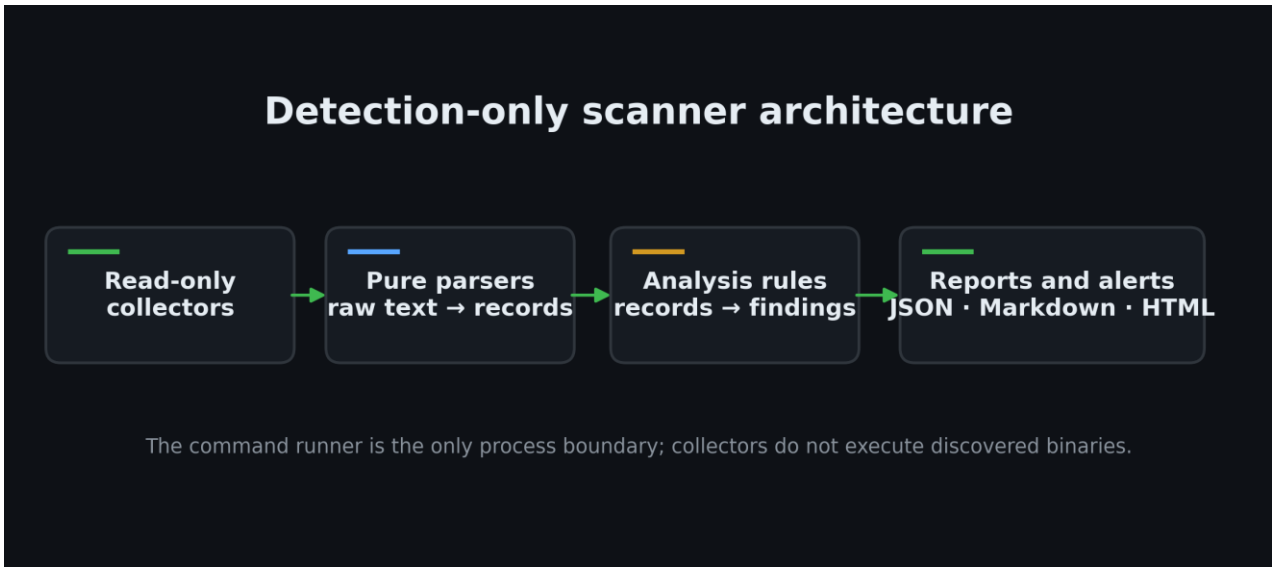


Figure 1. Scanner components and data flow

The separation keeps parsing tests platform-independent and makes rule behavior easy to explain. When a Linux command is absent, times out, or cannot read a path, the scan records a coverage limitation rather than treating missing evidence as a clean result.

Workflow

The scan follows the required order from system information through SUID discovery, permissions, services and sudo, cron, kernel checks, rule analysis, and report output. Each collector is independent, so one unavailable utility does not prevent other checks from completing.

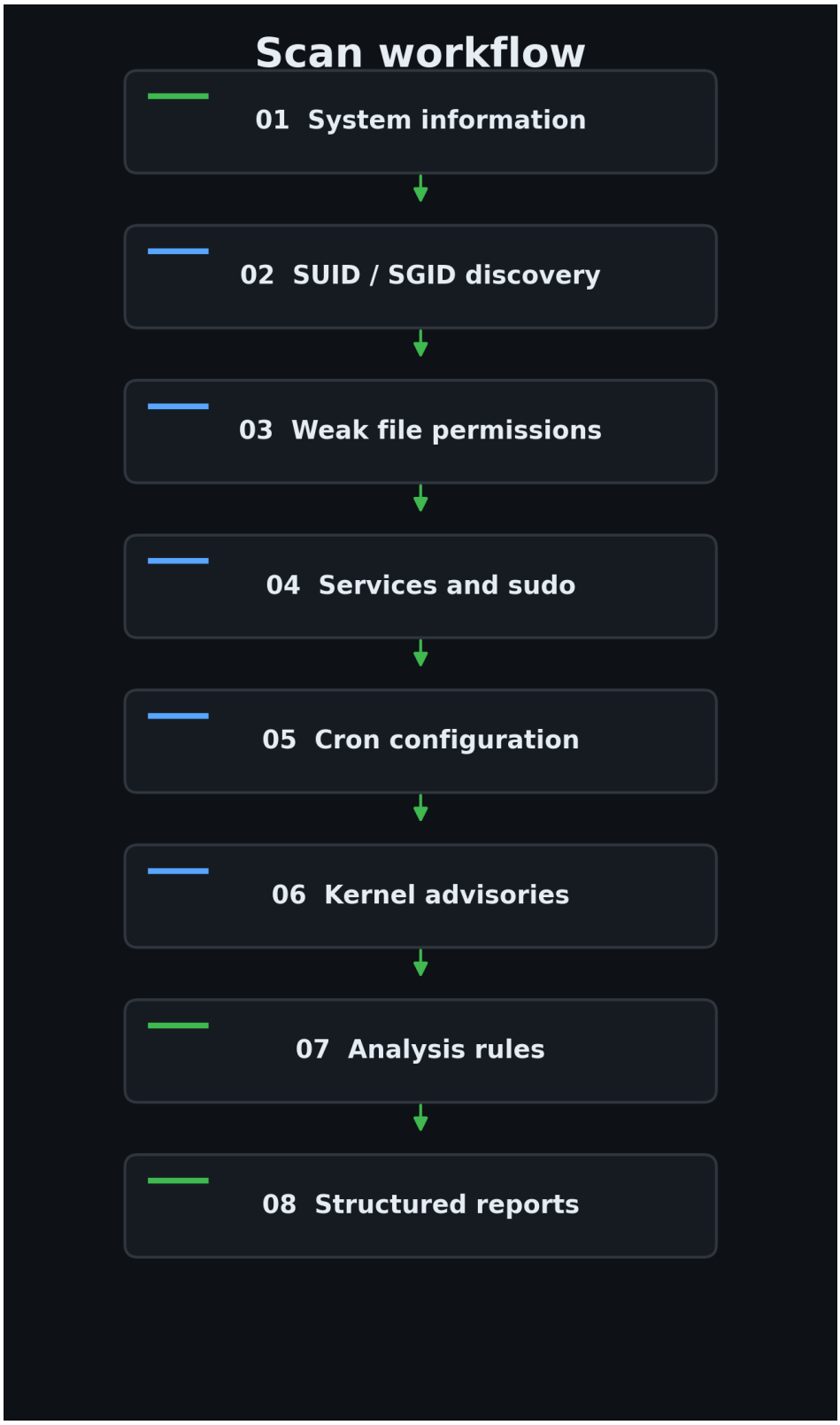


Figure 2. Read-only scan workflow

Implementation

Module	Plain-language responsibility
sysinfo	Records account, groups, host, OS release, kernel, Python, WSL/container markers, and privilege level.
suid	Uses find to list SUID/SGID files and parses mode, owner, group, and path.
perms	Checks world-writable items, sensitive configuration, home directories, and PATH directories.
services	Reads systemd unit files and parses root identity, executable, and PATH environment.
sudo	Reads sudoers when allowed or uses non-interactive sudo listing without prompting.
cron	Parses system schedules and records root job scripts, wildcard patterns, and relative commands.
caps	Uses getcap output to flag capabilities with a clear privilege impact.
kernel	Parses version prefixes and applies explicit upstream advisory ranges and an EOL policy.
report / compare	Writes structured reports and reports NEW, FIXED, and UNCHANGED fingerprints.

Collectors are read-only. Their outputs contain metadata and relevant configuration evidence, while the analysis layer decides whether a record matches a rule. This keeps command output from being treated as an instruction or executed as code.

Rule catalog

Rule IDs	Check	Default severity
SUID-001 / SUID-002	Non-standard SUID/SGID and known high-risk binary contexts	High / Medium
PERM-001 / PERM-002	World-writable files and directories, with sticky temporary-directory guard	High
PERM-003 / PERM-004 / PERM-005	Sensitive config, writable home directories, and writable PATH entries	Critical / High
SUDO-001 / SUDO-002 / SUDO-003	Passwordless rules, broad command grants, and GTFOBins contexts	High / Critical
CRON-001 / CRON-004	Root jobs whose scripts, parent directories, or schedule files are user-controlled	Critical
CRON-002 / CRON-003	Wildcard archive commands and relative root command paths	High / Medium
SVC-001 / SVC-002 / SVC-003	Writable root service units, executables, and unsafe PATH entries	Critical / High
CAP-001 / CAP-002	Privilege-sensitive capabilities and capable general-purpose interpreters	High / Critical
KERN-001 / KERN-002	Verified advisory range overlap and an explicitly documented upstream EOL policy	High / Medium

Rules are deliberately small. A rule describes exposure, not successful exploitation. A GTFOBins link explains why a binary's context merits review; the project stores no payload strings. Kernel rules state their affected ranges and are advisory because distribution kernels may contain backported fixes.

Safety model

Allowlisted commands

runner.py accepts only the specified identity, find, getcap, crontab, sudo-listing, systemctl-listing, and GNU find version forms. It rejects shell operators, dangerous find actions, and interactive sudo forms.

Timeout and argument safety

Every external invocation uses a list of arguments, shell=False, and a positive timeout. A timeout or unavailable tool becomes a skipped coverage record.

No mutation

The scanner does not call permission, ownership, service, file deletion, or account-changing APIs. It writes only reports, logs, and explicitly selected output folders.

No payloads

Findings state impact in ordinary language and link to references. The rule catalog contains names and URLs but no exploit commands or proof-of-concept code.

Runtime proof

The lab demo compares hashes of /etc/passwd, /etc/group, /etc/shadow, and /etc/sudoers around every scan. Unit tests also reject disallowed commands and scan implementation source for mutation patterns.

The separate lab intentionally creates weak configurations in a disposable guest only. It requires root plus the explicit confirmation flag, refuses pre-existing paths, writes a manifest, and removes only manifest-listed items. The scanner itself never invokes the lab scripts.

Lab design

Lab item	Harmless test condition	Expected rule family
SUID/SGID copies	Tracked copies of find and awk carry SUID bits; no copy is executed.	SUID-001, SUID-002
Permissions	A configuration file is mode 0666 and a directory is mode 0777.	PERM-001, PERM-002
Cron	Dormant yearly root entries reference a writable placeholder and a wildcard archive pattern.	CRON-001, CRON-002, CRON-003
Sudo	A lab-only sudoers drop-in grants a command and broad passwordless policy.	SUDO-001, SUDO-002, SUDO-003
systemd	An unstarted unit, target script, and PATH demonstrate metadata checks.	SVC-001, SVC-002, SVC-003
Capabilities	A copied interpreter receives cap_setuid when setcap is available.	CAP-001, CAP-002
Limited account	A locked lab account has a deliberately broad home mode.	PERM-004

No cron job is triggered and the service is never enabled or started. SUID and capability copies are inspected only as file metadata. Teardown reads the manifest, removes exact created files, removes the lab user and home, verifies absence, and is safe to run a second time.

Testing and verification

Verification	Observed result
pytest run 1	28 passed; 0 failed; 1 skipped
pytest run 2	28 passed; 0 failed; 1 skipped
Windows unsupported scan	PASS — unsupported host scan returned exit 2
Linux end-to-end scan	SKIPPED — guest control was unavailable before Kali login
Lab demo twice and compare fingerprints	SKIPPED — real Linux demo not available
Safety hash proof	SKIPPED — no Linux scan run
Independent cross-check and teardown	SKIPPED — no Linux lab run
Bash syntax and shellcheck	SKIPPED — no authenticated Linux shell session

The Windows fixture suite is evidence for portable parsing, rule logic, report output, comparison, and runner validation. It is not evidence that the Linux collectors work against a real host. The Linux end-to-end test is marked to skip on Windows, and this report keeps that distinction visible.

Results

Real Linux findings are SKIPPED. The Kali VM reported zero logged-in guest users, and VBoxManage guest-control returned that Guest Additions were not installed or not ready. No Act 1, Act 2, or Act 3 scan output exists, so no finding total, risk rating, screenshot, fixed count, or kernel result is stated here.

Requested result	Status
Act 1 clean baseline	SKIPPED — no authenticated guest session
Act 2 root and limited-user scans	SKIPPED — lab was not created
Simulated kernel report	SKIPPED — no Linux report exists
Act 3 teardown and before/after comparison	SKIPPED — no lab artifacts were created
Real finding screenshots	SKIPPED — no Linux transcript exists

Mitigation guidance

Finding area	Defensive response
SUID/SGID	Remove privilege bits from nonessential binaries, maintain package ownership, and review every non-standard path.
World-write access	Remove untrusted write bits from files and directories; use sticky mode only for shared temporary directories.
sudo	Replace broad command grants with exact, least-privilege rules and validate edits using visudo.
Cron	Make scripts and parent directories root-owned, use absolute paths, and pass explicit file lists to archive jobs.
systemd	Protect unit files and executables from user writes; use a short explicit PATH of trusted directories.
Capabilities	Remove unneeded capabilities and restrict required capabilities to a minimal protected helper.
Kernel	Apply the distribution's security update and verify vendor backport status before classifying a kernel version.

Mitigation advice is specific to the configuration class. Administrators should preserve application requirements, test changes in staging, and follow the distribution's own service and package procedures.

Limitations and future work

Current limitations

A version string cannot prove that a vendor kernel is vulnerable because distributions backport fixes. Files unreadable by a limited account remain unknown, not clean. GTFOBins matching is contextual and can produce findings that require human review. File scans may be incomplete when GNU find, getcap, or permissions are unavailable.

The current host had no configured WSL distribution and Docker's Linux engine was unavailable. VirtualBox showed Kali as running, but no user session was logged in and guest-control could not start a process. Therefore no real Linux scan or planted lab was executed in this build.

Future work

- Add distribution-specific kernel advisory sources with explicit vendor version parsing.
- Add a signed or checksum-verified rules data update process.
- Expand service and cron path resolution with more fixture-backed edge cases.
- Use the supplied Kali guest after login to collect two reproducible runs and refresh report metrics.

Ethics and legal note

Run the scanner only on systems you own or have written authorization to assess. Enumeration can reveal account names, file paths, and security configuration, so store reports as sensitive operational data and share them only with approved recipients.

The lab scripts create real local weaknesses and are restricted to disposable virtual machines or containers. Do not run them on a production server, personal daily-use computer, shared university system, or any environment whose owner has not approved the exercise. Finish the demo with teardown and verify the manifest-listed paths are absent.

This project provides defensive triage guidance, not a guarantee that a system is secure. A clean report means only that the enabled checks did not match their documented patterns with the available visibility.

Conclusion

The toolkit separates safe collection, pure parsing, small analysis rules, and report output. Its command runner, timeouts, coverage records, advisory references, and safety tests make the boundary between detection and exploitation explicit.

The fixture-based suite runs on Windows and supports early development without pretending that Windows is Linux. Real system validation remains a separate step. The included lab and exact Linux commands provide a repeatable path for collecting that evidence later, after an authenticated Linux shell is available.

Before submission, replace the cover placeholders, review the generated speaker notes, run the Kali demo twice, verify the teardown and hash proof, and update this report from those final evidence files.

References

NIST National Vulnerability Database, CVE-2016-5195 (Dirty COW): https://nvd.nist.gov/vuln/detail/CVE-2016-5195

NIST National Vulnerability Database, CVE-2022-0847 (Dirty Pipe): https://nvd.nist.gov/vuln/detail/CVE-2022-0847

The Linux Kernel Archives, Active kernel releases: https://www.kernel.org/releases.html

GTFOBins, binary behavior reference: https://gtfobins.github.io/

MITRE ATT&CK T1548.001, Setuid and Setgid: https://attack.mitre.org/techniques/T1548/001/

MITRE ATT&CK T1548.003, Sudo and Sudo Caching: https://attack.mitre.org/techniques/T1548/003/

MITRE ATT&CK T1053.003, Cron: https://attack.mitre.org/techniques/T1053/003/

MITRE ATT&CK T1543.002, Systemd Service: https://attack.mitre.org/techniques/T1543/002/

MITRE ATT&CK T1068, Exploitation for Privilege Escalation: https://attack.mitre.org/techniques/T1068/

Kernel range and support data were checked against NVD and kernel.org on 2026-11-09. The scanner's version-only matches are advisory and require vendor validation.

Appendix A: Command-line reference

```
python3 privesc_scan.py scan [--modules sysinfo,suid,perms,services,sudo,cron,caps,kernel] [--fast] [--exclude PATH] [--timeout SECONDS] [--label NAME] [--output-dir reports] [--format json,md,html] [--kernel-override VERSION] [--no-color]
python3 privesc_scan.py compare OLD.json NEW.json [--output-dir reports]
python3 privesc_scan.py render SCAN.json --format md,html
bash lab/run_lab_demo.sh --out evidence/run1 --pace 3
```

A kernel override is for demonstrations only and is visibly marked in console output and the JSON, Markdown, and HTML report. On non-Linux systems, help, compare, and render remain available; scan exits with status 2.

Appendix B: JSON structure

The JSON document contains schema_version, tool, created_at, meta, summary, findings, coverage, and commands. Each finding contains an ID, category, title, severity, confidence, evidence, exploitation possibility, mitigation, references, module, and stable fingerprint.

```
{
  "schema_version": 1,
  "meta": { "label": "<scan label>", "kernel": "<observed version>" },
  "summary": { "total": "<computed from evidence>" },
  "findings": [ "<records from the scan; omitted when no run exists>" ],
  "coverage": { "<check>": { "status": "ran|skipped", "reason": "..." } },
  "commands": [ "<allowlisted commands and observed outcomes>" ]
}
```

The values in angle brackets above describe the schema only. They are not scanner output or evidence.